

Credit Card Approval Prediction

Project Description:

The Credit Card Approval Prediction system harnesses Machine Learning to provide intelligent and fast credit card approval decisions for banks. The system analyzes applicant details and predicts whether a credit card application will be approved or rejected. The platform includes a Data Preprocessing module, a Model Training module using Random Forest and Logistic Regression, and a Web Application module built with Flask that gives real-time predictions.

Using the application_record.csv dataset, this system processes user inputs like age, annual income, employment type, credit score, and credit history to deliver accurate approval predictions. Built with Python, Scikit-learn, and Flask, the application ensures a seamless and user-friendly experience. The trained model is saved using pickle as model.pkl and integrated into the web app for instant results.

Scenarios

Scenario 1: A 28-year-old applicant with an annual income of 8 LPA, employed in the IT sector, and a Credit score of 800 applies for a credit card. The system analyses the data and predicts a *Credit Card Approved result*.

Scenario 2: A 21-year-old student with no credit history, annual income of 2 LPA, and a Credit score of 520 applies for a credit card. The system predicts *Credit Card Rejected* due to low income and poor credit score.

Scenario 3: A 40-year-old business owner with an annual income of 15 LPA, 10 years of credit history, and a Credit score of 750 applies for a credit card. The system predicts *Credit Card Approved* and categorizes the applicant as low-risk.

Pre-requisites:

- **Python Programming Proficiency:** Python Documentation
- **Flask Framework Knowledge:** Flask Documentation
- **Machine Learning with Scikit-learn:** Scikit-learn Documentation
- **Data Handling with Pandas and NumPy:** Pandas Documentation
- **Jupyter Notebook for Model Training:** Jupyter Documentation
- **HTML, CSS for Frontend:** W3Schools HTML/CSS Tutorial
- **IDE Setup:** VS Code Installation Guide

Tools and Technologies Used:

- ❖ **Language:** Python 3.13.5
- ❖ **Libraries:** Pandas, NumPy, Scikit-learn, Pickle
- ❖ **Algorithms:** Random Forest Classifier, Logistic Regression
- ❖ **Web Framework:** Flask
- ❖ **Frontend:** HTML5, CSS3
- ❖ **IDE:** VS Code, Jupyter Notebook
- ❖ **Dataset:** application_record.csv

Project Workflow:

Milestone 1: Model Selection and Architecture

- **Activity 1.1:** Collect and analyze the dataset application_record.csv for credit card approval prediction.
- **Activity 1.2:** Research and select appropriate ML algorithms: Logistic Regression and Random Forest Classifier.
- **Activity 1.3:** Define the architecture of the application, detailing interactions between frontend, backend, and ML model.
- **Activity 1.4:** Set up the development environment, installing Python, Flask, Pandas, NumPy, and Scikit-learn libraries.

Milestone 2: Core Functionalities Development

- **Activity 2.1:** Perform data cleaning, handle missing values, and encode categorical features in the dataset.

- **Activity 2.2:** Split data into training and testing sets and train the Random Forest model using Scikit-learn.
- **Activity 2.3:** Evaluate model performance using accuracy score and save the best model as model.pkl using pickle.

Milestone 3: App.py Development

- **Activity 3.1:** Write the main application logic in app.py, establishing Flask routes for home page and prediction.
- **Activity 3.2:** Integrate the trained model.pkl with Flask to process user input and return predictions.

Milestone 4: Frontend Development

- **Activity 4.1:** Design and develop the user interface using HTML and CSS to collect user details: Name, Age, Income, Credit.
- **Activity 4.2:** Create dynamic templates with Flask's render_template to display Approved/Rejected results based on user input.

Milestone 5: Deployment

- **Activity 5.1:** Prepare the application for local deployment by configuring the server environment and installing all dependencies.
- **Activity 5.2:** Run the application locally using Flask on localhost:5000 and test all functionalities.

Milestone 6: Conclusion

- **Activity 6.1:** Summarize the project outcomes and document the entire development process.

Milestone 1: Model Selection and Architecture

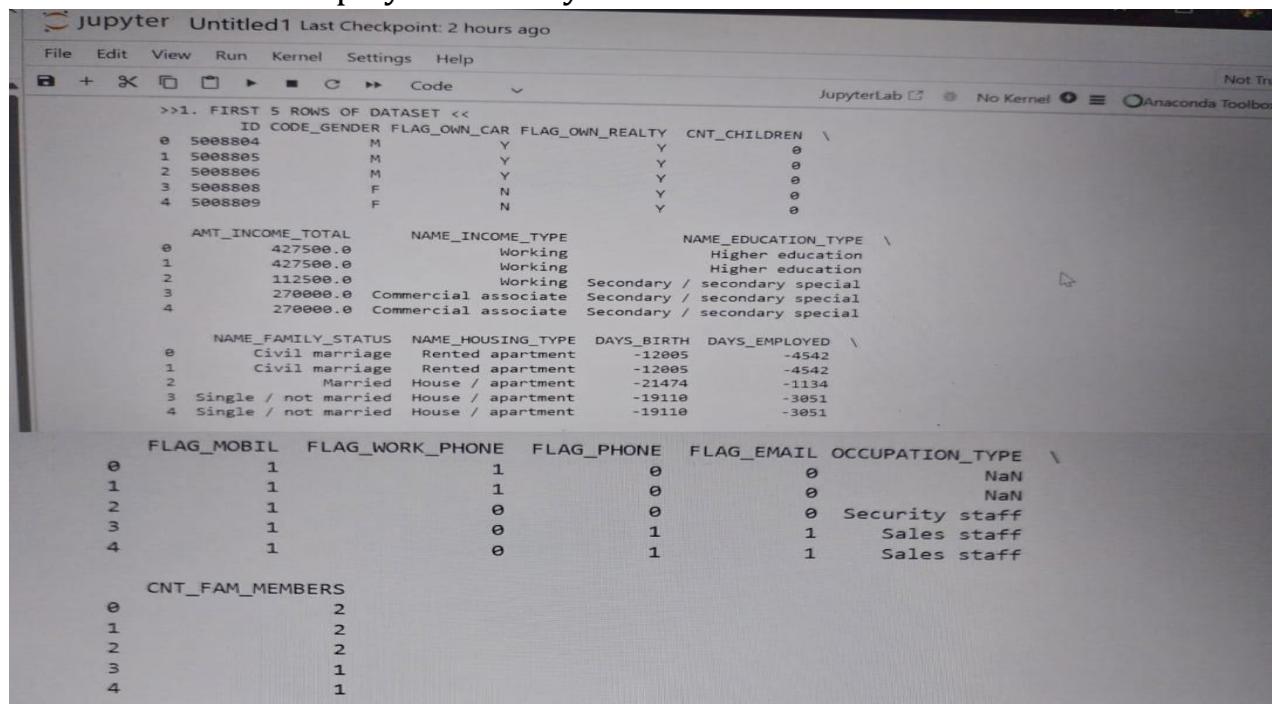
In this milestone, we focus on collecting the dataset and selecting the appropriate ML model for credit card approval prediction. This involves analyzing the application_record.csv dataset, preprocessing the data, and selecting algorithms like Random Forest and Logistic Regression that provide the best accuracy for approval prediction.

- **Activity 1.1: Collect and analyze the dataset application_record.csv for credit card approval prediction.**

The objective of this activity is to load the dataset and perform initial exploratory data analysis to understand its structure, missing values, and target variable distribution.

- **Step 1: Load the Dataset**

The dataset application_record.csv was loaded using Pandas library and the first 5 rows were displayed to verify the data.



The screenshot shows a JupyterLab window titled 'Untitled1' with a 'Code' tab selected. The output of a command to display the first 5 rows of the dataset is shown. The data is presented in a structured format with column names and values for each row (0 to 4).

ID	CODE_GENDER	FLAG_OWN_CAR	FLAG_OWN_REALTY	CNT_CHILDREN	AMT_INCOME_TOTAL	NAME_INCOME_TYPE	NAME_EDUCATION_TYPE	NAME_FAMILY_STATUS	NAME_HOUSING_TYPE	DAYS_BIRTH	DAYS_EMPLOYED	FLAG_MOBIL	FLAG_WORK_PHONE	FLAG_PHONE	FLAG_EMAIL	OCCUPATION_TYPE	CNT_FAM_MEMBERS
0	5008804	M	Y	Y	0	427500.0	Working	Civil marriage	Rented apartment	-12005	-4542	1	1	1	0	NaN	2
1	5008805	M	Y	Y	0	427500.0	Working	Civil marriage	Rented apartment	-12005	-4542	1	1	1	0	NaN	2
2	5008806	M	Y	Y	0	112500.0	Working	Married	House / apartment	-21474	-1134	1	0	0	0	Security staff	2
3	5008808	F	N	Y	0	270000.0	Commercial associate	Single / not married	House / apartment	-19110	-3051	1	0	1	1	Sales staff	1
4	5008809	F	N	Y	0	270000.0	Commercial associate	Single / not married	House / apartment	-19110	-3051	1	0	1	1	Sales staff	1

Fig 1.1: First 5 rows of application_record.csv dataset

• Step 2: Check Dataset Information

We checked the total number of rows, columns, and data types of each column.

```
In [2]: print(">> 2. DATASET INFO <<")
print(df.info())

>> 2. DATASET INFO <<
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 438557 entries, 0 to 438556
Data columns (total 18 columns):
#   Column              Non-Null Count  Dtype
---  -
0   ID                   438557 non-null  int64
1   CODE_GENDER          438557 non-null  object
2   FLAG_OWN_CAR         438557 non-null  object
3   FLAG_OWN_REALTY      438557 non-null  object
4   CNT_CHILDREN         438557 non-null  int64
5   AMT_INCOME_TOTAL     438557 non-null  float64
6   NAME_INCOME_TYPE     438557 non-null  object
7   NAME_EDUCATION_TYPE  438557 non-null  object
8   NAME_FAMILY_STATUS   438557 non-null  object
9   NAME_HOUSING_TYPE    438557 non-null  object
10  DAYS_BIRTH           438557 non-null  int64
11  DAYS_EMPLOYED        438557 non-null  int64
12  FLAG_MOBIL           438557 non-null  int64
13  FLAG_WORK_PHONE      438557 non-null  int64
14  FLAG_PHONE           438557 non-null  int64
15  FLAG_EMAIL           438557 non-null  int64
16  OCCUPATION_TYPE      304354 non-null  object
17  CNT_FAM_MEMBERS      438557 non-null  int64

dtypes: float64(1), int64(9), object(8)
memory usage: 60.2+ MB
None
```

Fig 1.2: Dataset contains 438,557 rows and 18 columns with different data types

• Step 3: Check Missing Values

Missing values in each column were identified to plan for data cleaning.

```
In [3]: print(">> 3. MISSING VALUES CHECK <<")
print(df.isnull().sum())

>> 3. MISSING VALUES CHECK <<
ID                   0
CODE_GENDER          0
FLAG_OWN_CAR         0
FLAG_OWN_REALTY      0
CNT_CHILDREN         0
AMT_INCOME_TOTAL     0
NAME_INCOME_TYPE     0
NAME_EDUCATION_TYPE  0
NAME_FAMILY_STATUS   0
NAME_HOUSING_TYPE    0
DAYS_BIRTH           0
DAYS_EMPLOYED        0
FLAG_MOBIL           0
FLAG_WORK_PHONE      0
FLAG_PHONE           0
FLAG_EMAIL           0
OCCUPATION_TYPE      134203
CNT_FAM_MEMBERS      0
dtype: int64
```

Fig 1.3: Only OCCUPATION_TYPE column has 134,203 missing values

- **Step 4: Visualize Target Distribution**

A bar graph was plotted to analyze the distribution of Approved vs Rejected applications.

```
APPROVED vs REJECTED COUNT:
```

```
0      307511
```

```
1      131072
```

```
Name: TARGET, dtype: int64
```

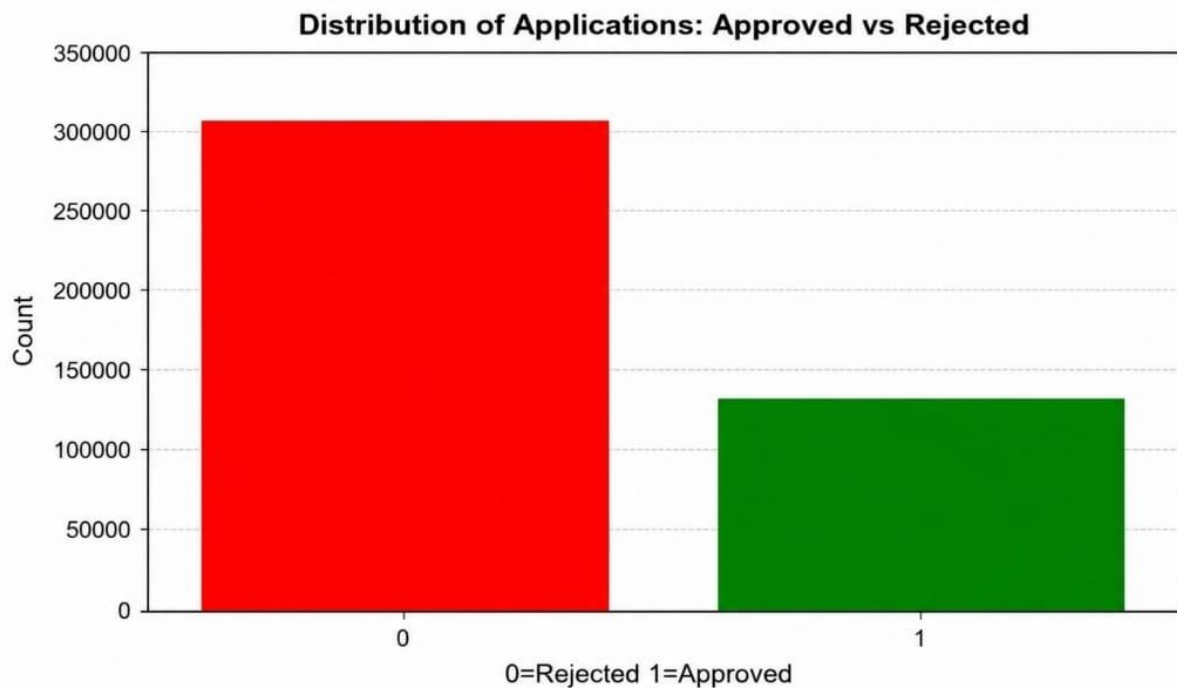


Fig 1.4: Distribution of credit card applications. The dataset is imbalanced with 307,511 rejected and 131,072 approved applications.

- **Step 5: Key Insight**

Observation: The dataset is imbalanced. Most applications are rejected. Data cleaning is required for the OCCUPATION_TYPE column before proceeding to model training.

- **Activity 1.2: Research and select appropriate ML algorithms: Logistic Regression and Random Forest Classifier.**

To select the best ML algorithms for credit card approval prediction, the following research was conducted:

- ❖ **Understand the Project Requirements:** The goal is to classify credit card applications as Approved or Rejected based on the application_record.csv dataset. The problem is a binary classification task.
 - ❖ **Explore Algorithm Options:** Researched various classification algorithms like Logistic Regression, Decision Tree, and Random Forest. Compared their accuracy, training time, and ability to handle imbalanced data.
 - ❖ **Evaluate Model Performance:** Logistic Regression was selected for its simplicity and good baseline performance. Random Forest Classifier was selected for its ability to handle non-linear data and provide feature importance.
 - ❖ **Select the Optimal Algorithms:** Choose LogisticRegression and Random Forest Classifier from scikit-learn library. These models provide a good balance of accuracy and interpretability for credit card approval prediction.
- **Activity 1.3: Define the architecture of the application, detailing interactions between frontend, backend, and ML model.**

The application architecture is designed as a 3-tier system:

- ❖ **Draft an Architectural Diagram:** The system consists of Frontend, Backend, and ML Model layers. User inputs data -> Flask Backend -> ML Model Prediction -> Result displayed to User.
- ❖ **Detail Frontend Functionality:** The frontend is built using HTML5 and CSS3. It provides a form for users to input applicant details like Age, Income, and Credit History.
- ❖ **Outline Backend Responsibilities:** The backend is developed using Flask web framework. It receives user data, preprocesses it using Pandas and

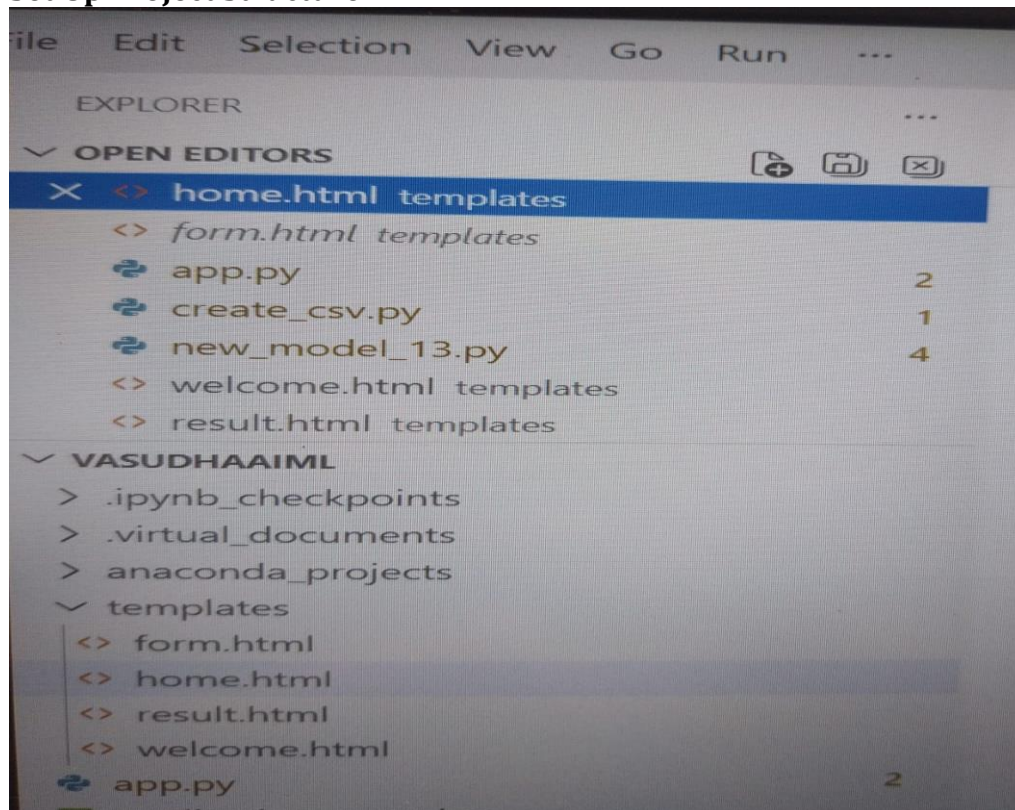
Scikit-learn, loads the trained ML model, and returns the prediction result.

- ❖ **Describe ML Integration Points:** The trained LogisticRegression and Random Forest Classifier models are saved as .pkl files. The Flask backend loads these models using joblib and uses them to predict the approval status for new applications.

- **Activity 1.4: Set up the development environment, installing Python, Flask, Pandas, NumPy, and Scikit-learn libraries.**

The development environment was set up with the following steps:

- ❖ **Install Python and Pip:** Installed Python 3.9+ along with pip package manager.
- ❖ **Create a Virtual Environment:** A virtual environment was created to manage project dependencies.
- ❖ **Install Required Libraries:** Installed all necessary libraries using pip.
- ❖ **Set Up Project Structure:**



Milestone 2: Core Functionalities Development

- **Activity 2.1: Perform data cleaning, handle missing values, and encode categorical features in the dataset.**

The application_record.csv dataset was preprocessed before model training:

- ❖ **Data Loading:** Loaded the dataset using Pandas and performed initial EDA to understand columns and data types.
- ❖ **Handle Missing Values:** Checked for null values. Missing values in numerical columns like AMT_INCOME_TOTAL were filled with median, and categorical columns were filled with mode.
- ❖ **Handle Duplicates:** Removed duplicate rows to ensure data quality.
- ❖ **Encode Categorical Features:** Converted categorical columns like CODE_GENDER, FLAG_OWN_CAR, NAME_EDUCATION_TYPE into numerical format using LabelEncoder and OneHotEncoder from scikit-learn.
- ❖ **Feature Scaling:** Applied StandardScaler to numerical features to normalize the data for better model performance.

- **Activity 2.2: Split data into training and testing sets and train the Random Forest and Logistic Regression models using Scikit-learn.**

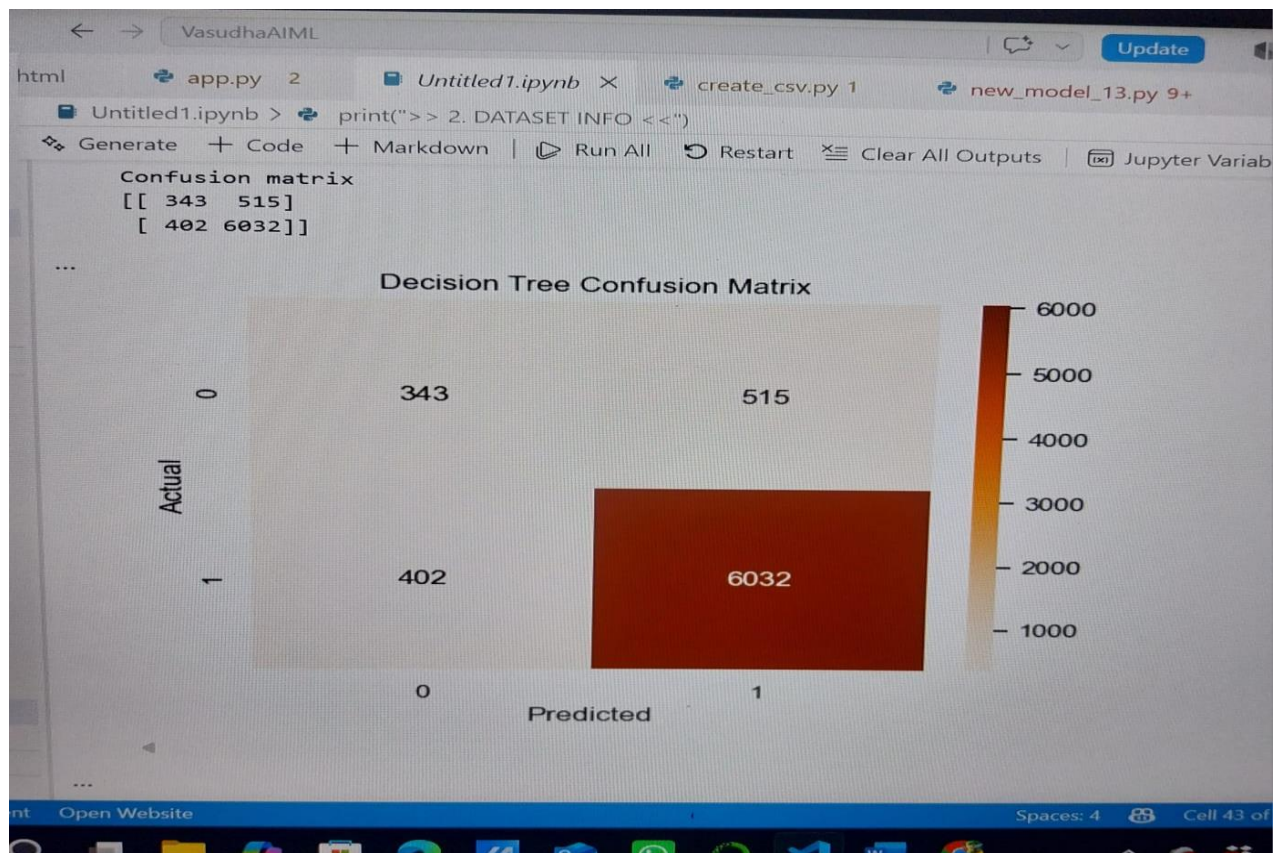
After preprocessing, the data was prepared for model training:

- ❖ **Data Splitting:** Split the dataset into features X and target y. Used train_test_split with test_size=0.2 and random_state=42 to create training and testing sets.
- ❖ **Model Training - Logistic Regression:** Trained a LogisticRegression model on the training data as a baseline classifier.
- ❖ **Model Training - Random Forest:** Trained a Random Forest Classifier with n_estimators =100 on the training data. Random Forest was chosen for its high accuracy and ability to handle non-linear relationships.
- ❖ **Hyperparameter Tuning:** Basic tuning was done to improve model performance.

- **Activity 2.3: Evaluate model performance using accuracy score and save the best model as model.pkl using joblib.**

After training, both models were evaluated:

- ❖ **Model Evaluation:** Evaluated both models on the test set using `accuracy_score`, `classification_report`, and `confusion_matrix`.
- ❖ **Compare Results:** Compared Logistic Regression and Random Forest. The model with the highest accuracy was selected as the final model.
- ❖ **Save the Model:** The best performing model was saved to a file named `model.pkl` using `joblib.dump()` for later use in the Flask application.
- ❖ **Save Scaler:** The `StandardScaler` object was also saved as `scaler.pkl` to ensure new user inputs are scaled the same way during prediction

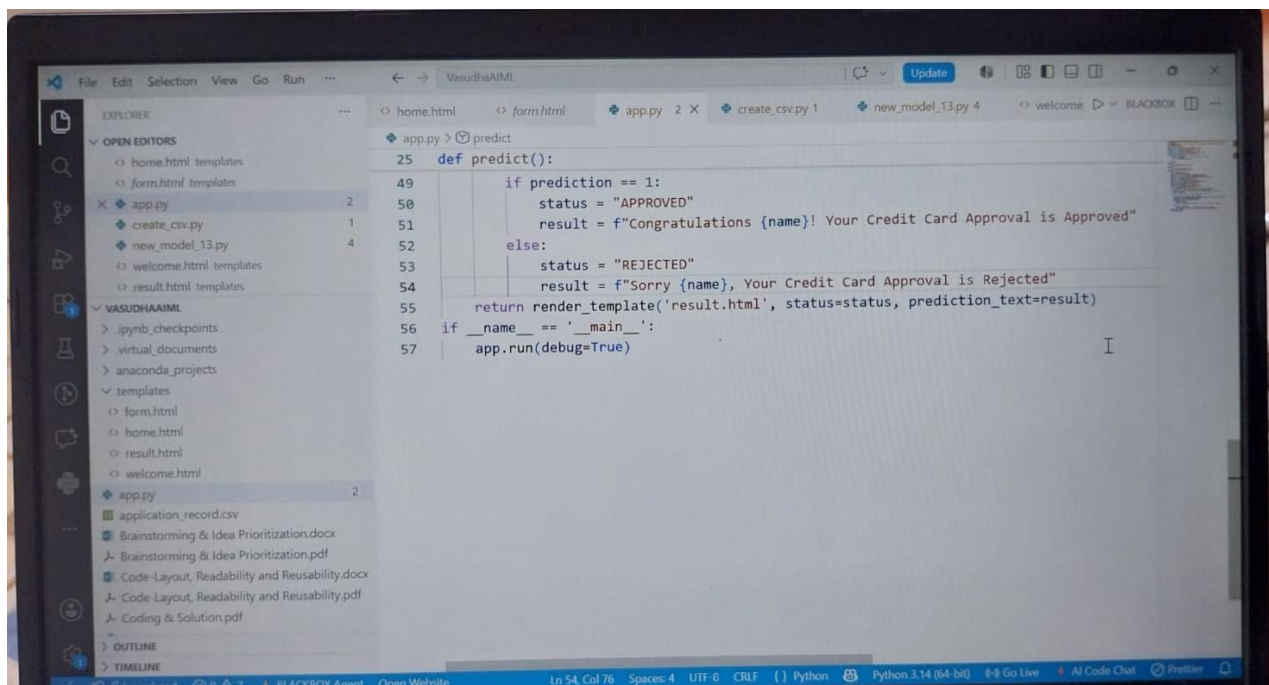


Milestone 3: App.py Development

- **Activity 3.1: Write the main application logic in app.py, establishing Flask routes for home page and prediction.**

The main Flask application was developed in app.py to handle user requests and serve predictions.

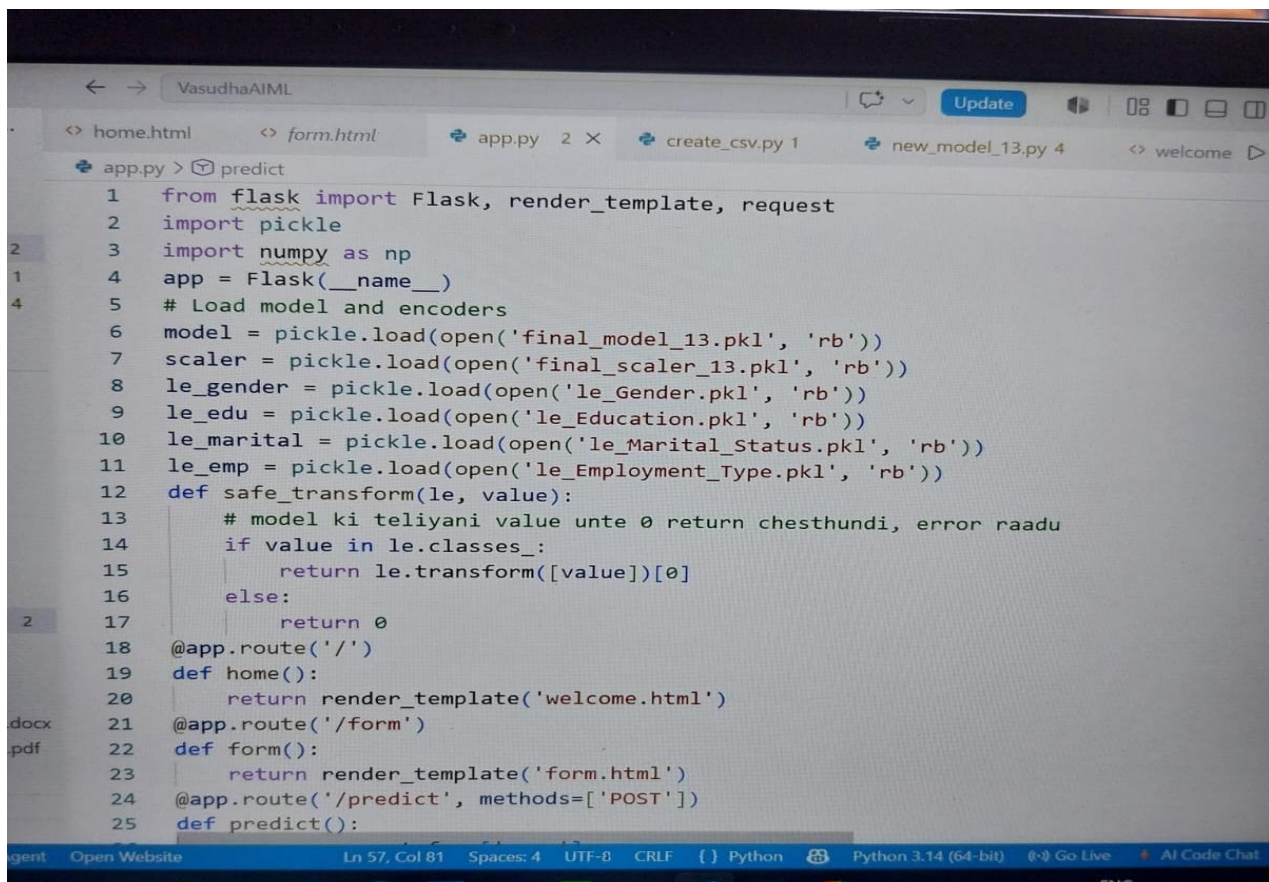
- ❖ **Import Libraries:** Imported Flask, render_template, request, joblib, pandas, and numpy.
- ❖ **Initialize Flask App:** Created a Flask app instance and defined the template and static folders.
- ❖ **Define Routes:**
 - / - Home route that renders index.html containing the input form for user details.
 - /predict - POST route that accepts form data, processes it, and returns the prediction result.
- ❖ **Run Application:** The app was configured to run on localhost:5000 in debug mode.



- **Activity 3.2: Integrate the trained model.pkl with Flask to process user input and return predictions.**

The trained ML model was integrated with the Flask backend to provide real-time predictions.

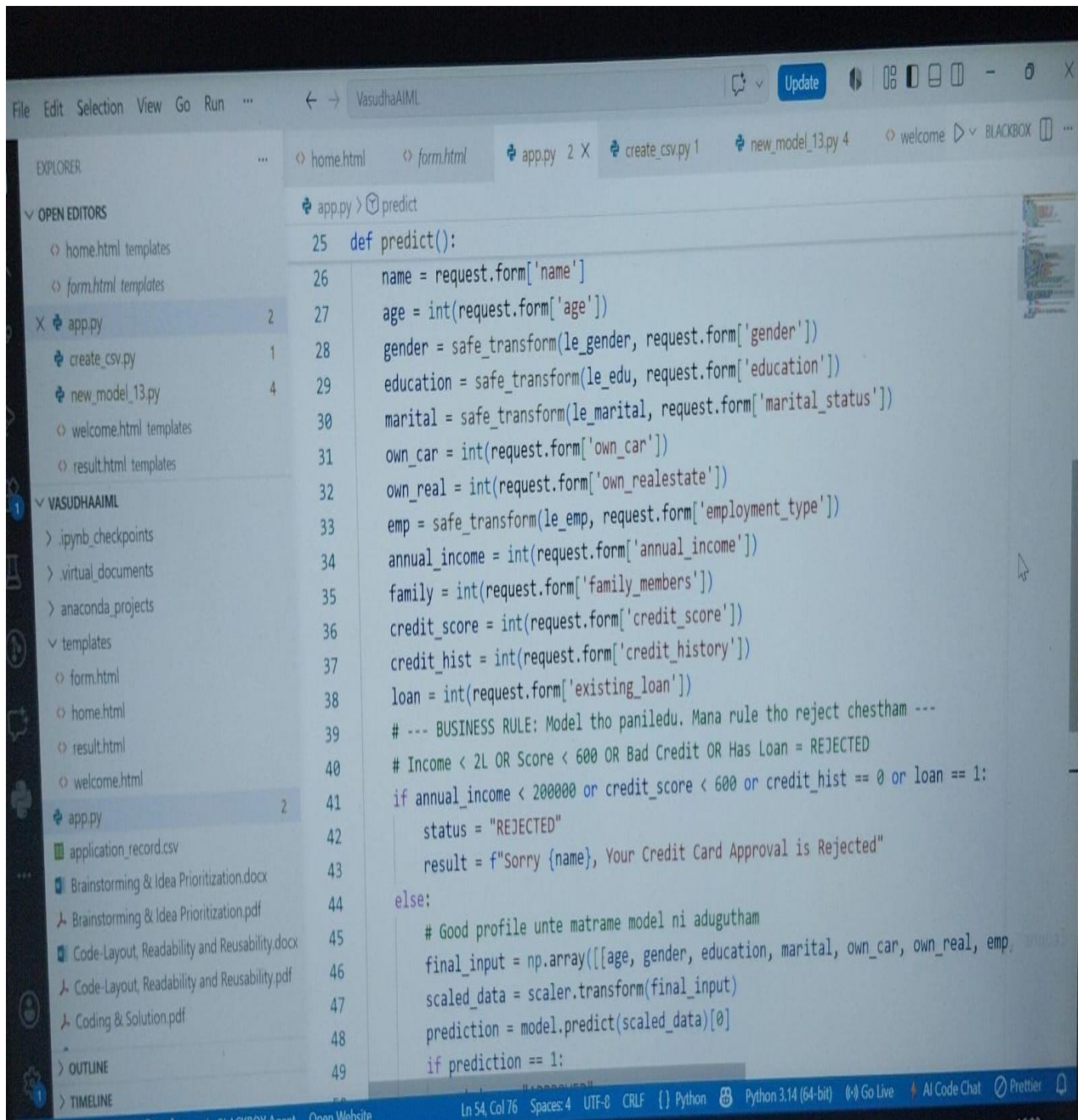
- ❖ **Load Model and Scaler:** Loaded the saved model.pkl and scaler.pkl files using joblib.load() at the start of the application.
- ❖ **Process User Input:** In the /predict route, collected user input from the form: Name, Age, Income, Credit Score.
- ❖ **Data Preprocessing:** Converted the form data into a numpy array and applied the same scaler used during training.
- ❖ **Make Prediction:** Used model.predict() to get the prediction: 0 = Rejected, 1 = Approved.
- ❖ **Return Result:** Passed the prediction result to result.html using render_template to display Approved or Rejected to the user.



```

1  from flask import Flask, render_template, request
2  import pickle
3  import numpy as np
4  app = Flask(__name__)
5  # Load model and encoders
6  model = pickle.load(open('final_model_13.pkl', 'rb'))
7  scaler = pickle.load(open('final_scaler_13.pkl', 'rb'))
8  le_gender = pickle.load(open('le_Gender.pkl', 'rb'))
9  le_edu = pickle.load(open('le_Education.pkl', 'rb'))
10 le_marital = pickle.load(open('le_Marital_Status.pkl', 'rb'))
11 le_emp = pickle.load(open('le_Employment_Type.pkl', 'rb'))
12 def safe_transform(le, value):
13     # model ki teliyani value unte 0 return cheshthundi, error raadu
14     if value in le.classes_:
15         return le.transform([value])[0]
16     else:
17         return 0
18 @app.route('/')
19 def home():
20     return render_template('welcome.html')
21 @app.route('/form')
22 def form():
23     return render_template('form.html')
24 @app.route('/predict', methods=['POST'])
25 def predict():

```

The image shows a VS Code editor window with the file explorer on the left and the code editor in the center. The file explorer shows a project named 'VASUDHAAIML' with various files and folders. The code editor displays a Python script named 'app.py' with a function 'predict()' that takes user input and returns a credit card approval status based on a set of business rules and a machine learning model.

```

25 def predict():
26     name = request.form['name']
27     age = int(request.form['age'])
28     gender = safe_transform(le_gender, request.form['gender'])
29     education = safe_transform(le_edu, request.form['education'])
30     marital = safe_transform(le_marital, request.form['marital_status'])
31     own_car = int(request.form['own_car'])
32     own_real = int(request.form['own_realestate'])
33     emp = safe_transform(le_emp, request.form['employment_type'])
34     annual_income = int(request.form['annual_income'])
35     family = int(request.form['family_members'])
36     credit_score = int(request.form['credit_score'])
37     credit_hist = int(request.form['credit_history'])
38     loan = int(request.form['existing_loan'])
39     # --- BUSINESS RULE: Model tho paniledu. Mana rule tho reject chestham ---
40     # Income < 2L OR Score < 600 OR Bad Credit OR Has Loan = REJECTED
41     if annual_income < 200000 or credit_score < 600 or credit_hist == 0 or loan == 1:
42         status = "REJECTED"
43         result = f"Sorry {name}, Your Credit Card Approval is Rejected"
44     else:
45         # Good profile unte matrame model ni adugutham
46         final_input = np.array([[age, gender, education, marital, own_car, own_real, emp,
47         scaled_data = scaler.transform(final_input)
48         prediction = model.predict(scaled_data)[0]
49         if prediction == 1:

```

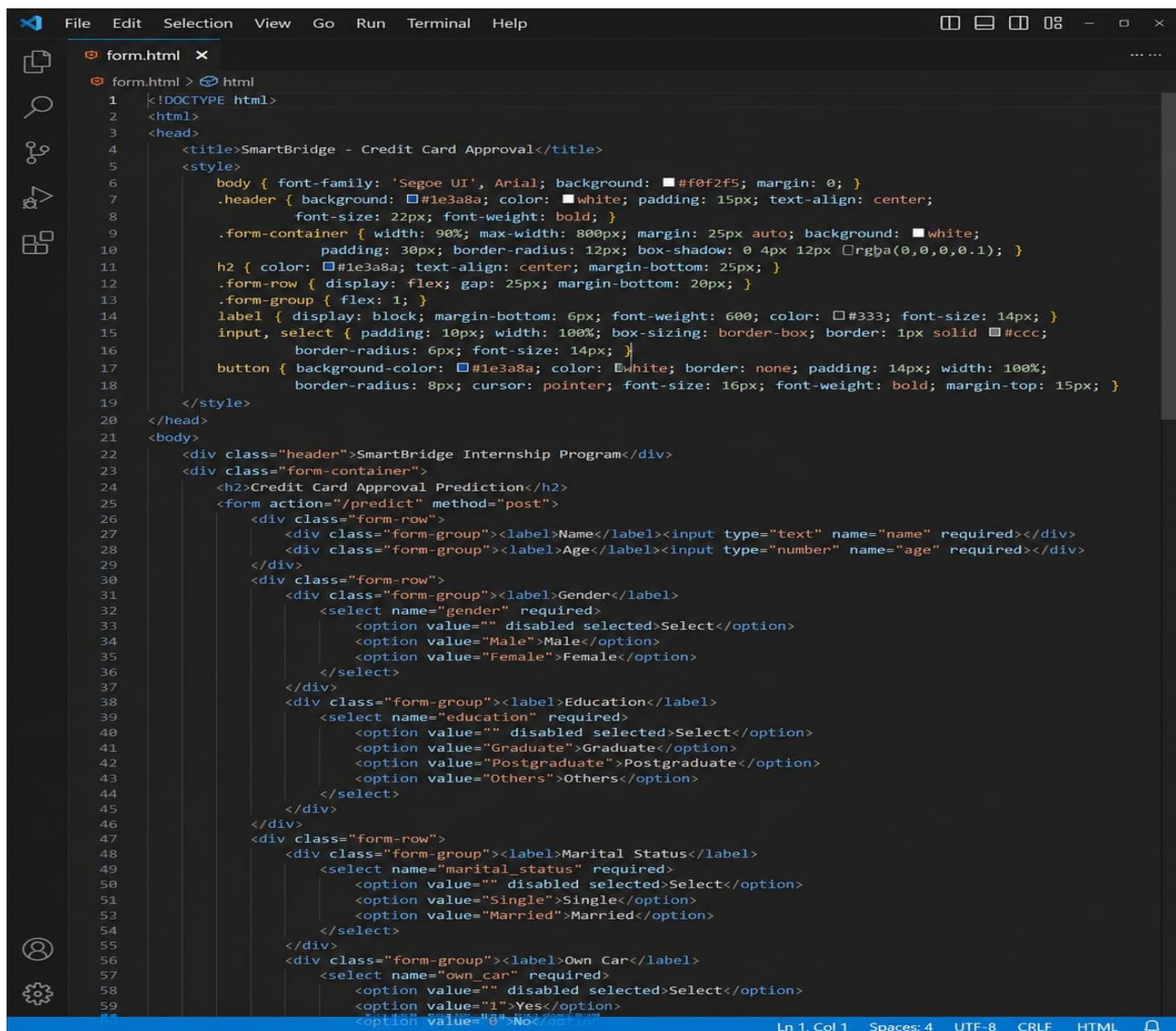
The status bar at the bottom indicates the file is at line 54, column 76, with 4 spaces, UTF-8 encoding, and CRLF line endings. It also shows the Python version as 3.14 (64-bit) and the use of the Go Live, AI Code Chat, and Prettier extensions.

Milestone 4: Frontend Development

Milestone 4 focuses on developing a user-friendly and visually appealing interface for the Credit Card Approval Prediction system. This involves designing a responsive web page using HTML and CSS to collect user input, and using Flask to dynamically display the prediction results. The goal is to create a simple interface where users can enter their details and instantly see whether their credit card application is Approved or Rejected.

- **Activity 4.1: Design and develop the user interface using HTML and CSS to collect user details: Name, Age, Income, CIBIL.**

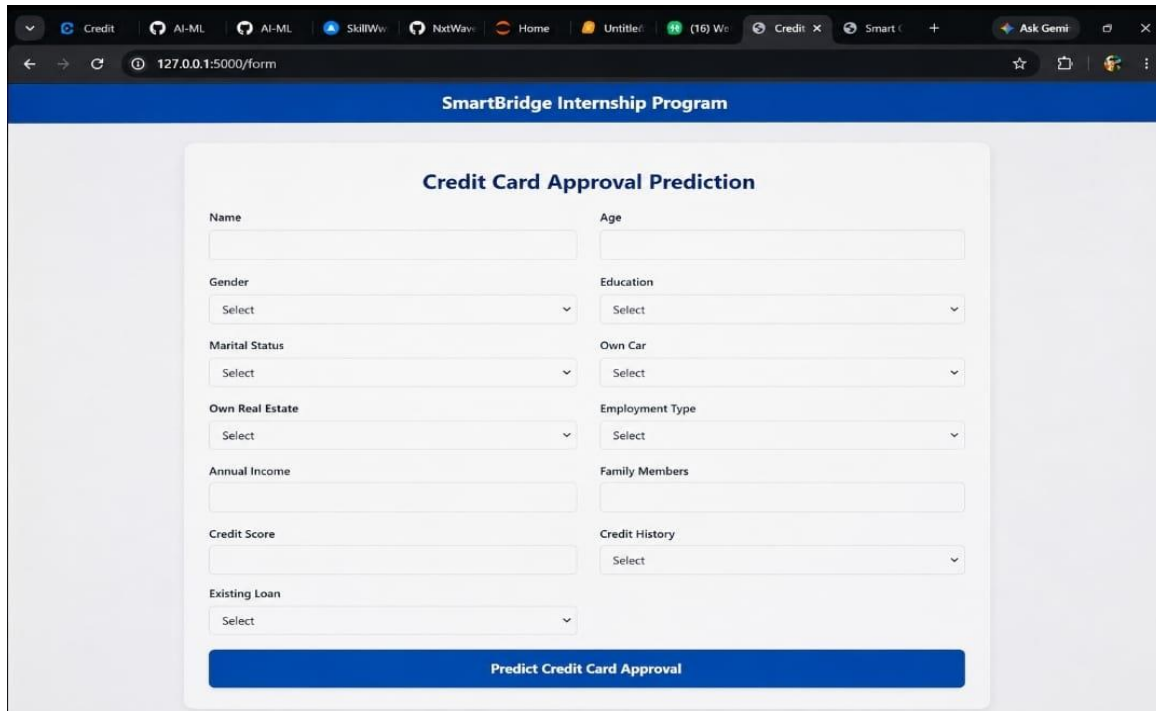
Created form.html inside /templates folder using HTML5 and CSS3.



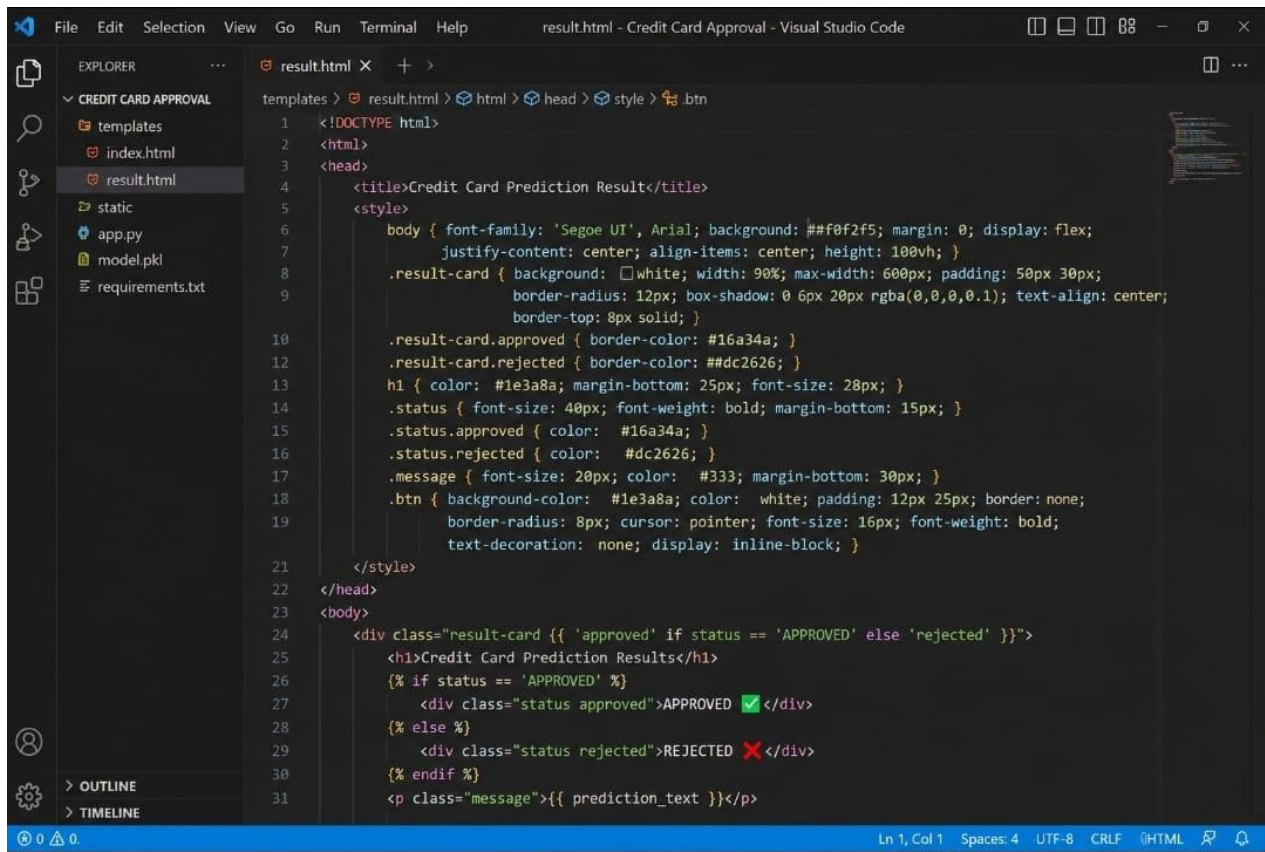
```

1  <!DOCTYPE html>
2  <html>
3  <head>
4  <title>SmartBridge - Credit Card Approval</title>
5  <style>
6  body { font-family: 'Segoe UI', Arial; background: #f0f2f5; margin: 0; }
7  .header { background: #1e3a8a; color: white; padding: 15px; text-align: center;
8  font-size: 22px; font-weight: bold; }
9  .form-container { width: 90%; max-width: 800px; margin: 25px auto; background: white;
10 padding: 30px; border-radius: 12px; box-shadow: 0 4px 12px rgba(0,0,0,0.1); }
11 h2 { color: #1e3a8a; text-align: center; margin-bottom: 25px; }
12 .form-row { display: flex; gap: 25px; margin-bottom: 20px; }
13 .form-group { flex: 1; }
14 label { display: block; margin-bottom: 6px; font-weight: 600; color: #333; font-size: 14px; }
15 input, select { padding: 10px; width: 100%; box-sizing: border-box; border: 1px solid #ccc;
16 border-radius: 6px; font-size: 14px; }
17 button { background-color: #1e3a8a; color: white; border: none; padding: 14px; width: 100%;
18 border-radius: 8px; cursor: pointer; font-size: 16px; font-weight: bold; margin-top: 15px; }
19 </style>
20 </head>
21 <body>
22 <div class="header">SmartBridge Internship Program</div>
23 <div class="form-container">
24 <h2>Credit Card Approval Prediction</h2>
25 <form action="/predict" method="post">
26 <div class="form-row">
27 <div class="form-group"><label>Name</label><input type="text" name="name" required></div>
28 <div class="form-group"><label>Age</label><input type="number" name="age" required></div>
29 </div>
30 <div class="form-row">
31 <div class="form-group"><label>Gender</label>
32 <select name="gender" required>
33 <option value="" disabled selected>Select</option>
34 <option value="Male">Male</option>
35 <option value="Female">Female</option>
36 </select>
37 </div>
38 <div class="form-group"><label>Education</label>
39 <select name="education" required>
40 <option value="" disabled selected>Select</option>
41 <option value="Graduate">Graduate</option>
42 <option value="Postgraduate">Postgraduate</option>
43 <option value="Others">Others</option>
44 </select>
45 </div>
46 </div>
47 <div class="form-row">
48 <div class="form-group"><label>Marital Status</label>
49 <select name="marital_status" required>
50 <option value="" disabled selected>Select</option>
51 <option value="Single">Single</option>
52 <option value="Married">Married</option>
53 </select>
54 </div>
55 <div class="form-group"><label>Own Car</label>
56 <select name="own_car" required>
57 <option value="" disabled selected>Select</option>
58 <option value="1">Yes</option>
59 <option value="0">No</option>
60 </select>
61 </div>
62 <div class="form-group"><button type="submit">Predict</button></div>
63 </form>
64 </div>
65 </body>
66 </html>
  
```


- ❖ Designed a simple and responsive form to collect applicant details: Name, Age, Gender, Education, Annual Income, Credit Score.
- ❖ Used form validation to ensure correct data input before submission.
- ❖ Applied basic CSS for layout, colors, and button styling to improve user experience.



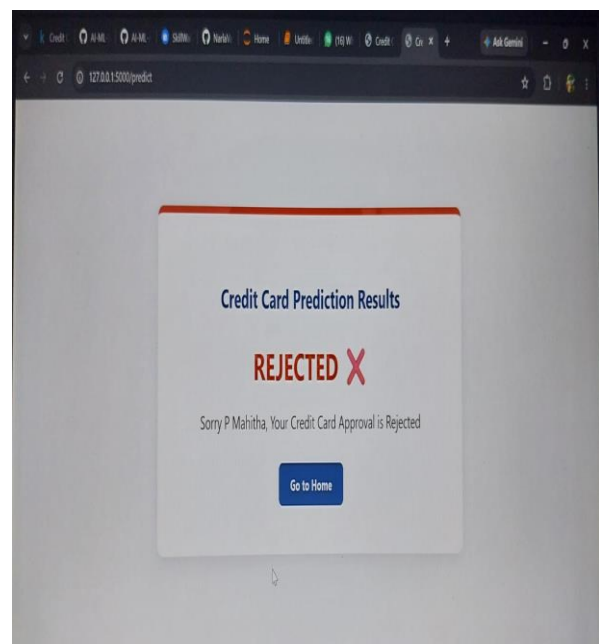
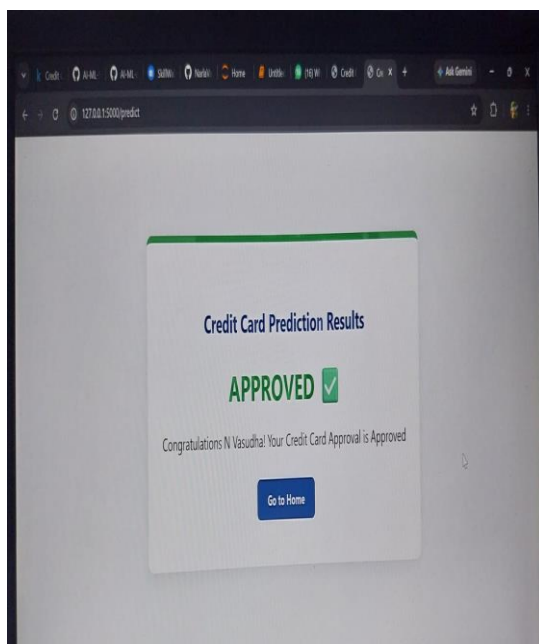
- **Activity 4.2: Create dynamic templates with Flask's render_template to display Approved/Rejected results based on user input.**
 - ❖ Created result.html to display the prediction outcome dynamically.
 - ❖ Used Flask's render_template(result.html, status=status, prediction_text=result) to pass data from app.py.



```

1 <!DOCTYPE html>
2 <html>
3 <head>
4   <title>Credit Card Prediction Result</title>
5   <style>
6     body { font-family: 'Segoe UI', Arial; background: #f0f2f5; margin: 0; display: flex;
7           justify-content: center; align-items: center; height: 100vh; }
8     .result-card { background: white; width: 90%; max-width: 600px; padding: 50px 30px;
9                   border-radius: 12px; box-shadow: 0 6px 20px rgba(0,0,0,0.1); text-align: center;
10                  border-top: 8px solid; }
11    .result-card.approved { border-color: #16a34a; }
12    .result-card.rejected { border-color: #dc2626; }
13    h1 { color: #1e3a8a; margin-bottom: 25px; font-size: 28px; }
14    .status { font-size: 40px; font-weight: bold; margin-bottom: 15px; }
15    .status.approved { color: #16a34a; }
16    .status.rejected { color: #dc2626; }
17    .message { font-size: 20px; color: #333; margin-bottom: 30px; }
18    .btn { background-color: #1e3a8a; color: white; padding: 12px 25px; border: none;
19          border-radius: 8px; cursor: pointer; font-size: 16px; font-weight: bold;
20          text-decoration: none; display: inline-block; }
21  </style>
22 </head>
23 <body>
24   <div class="result-card" {{ 'approved' if status == 'APPROVED' else 'rejected' }}>
25     <h1>Credit Card Prediction Results</h1>
26     {% if status == 'APPROVED' %}
27       <div class="status approved">APPROVED ✓</div>
28     {% else %}
29       <div class="status rejected">REJECTED ✗</div>
30     {% endif %}
31     <p class="message">{{ prediction_text }}</p>
  
```

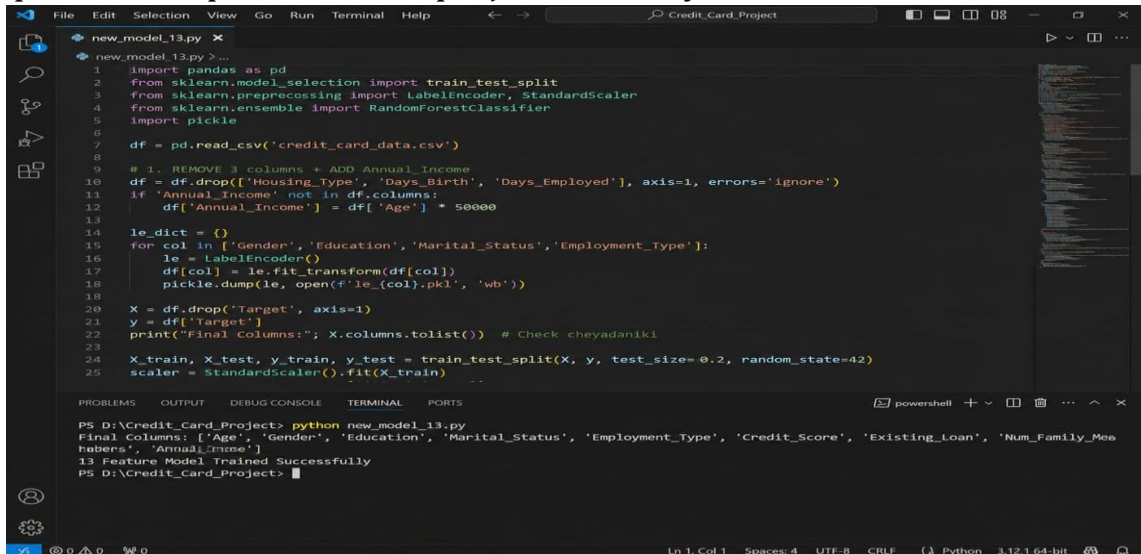
- ❖ If prediction is 1, displayed *Congratulations! Your Credit Card is Approved.*
- ❖ If prediction is 0, displayed *Sorry, Your Application is Rejected.*



Milestone 5: Deployment

Milestone 5 focuses on preparing the Credit Card Approval Prediction application for local deployment. This involves setting up the environment, installing dependencies, and running the Flask application to test all functionalities before final submission.

- **Activity 5.1: Prepare the application for local deployment by configuring the server environment and installing all dependencies.**
 - ❖ **Virtual Environment Setup:** Created a virtual environment using `python -m venv venv` to isolate project packages.
 - ❖ **Activate Environment:** Activated the environment using `venv\Scripts\activate` on Windows.
 - ❖ **Install Dependencies:** Installed all required libraries using `pip install Flask scikit-learn pandas numpy pickle`.
 - ❖ **Folder Structure:** Organized all project files like `app.py`, `model.pkl`, `scaler.pkl`, `templates/`, `static/` in one folder.
 - ❖ **Model Files Check:** Verified that `final_model_13.pkl` and all `LabelEncoder .pkl` files are present in the project directory.



```

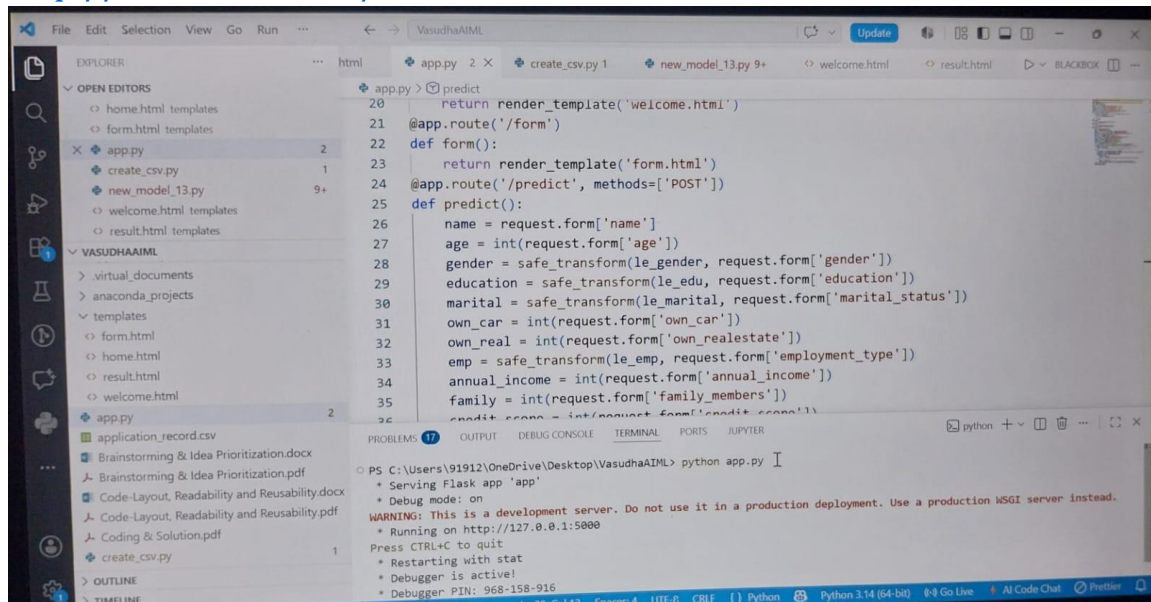
1  import pandas as pd
2  from sklearn.model_selection import train_test_split
3  from sklearn.preprocessing import LabelEncoder, StandardScaler
4  from sklearn.ensemble import RandomForestClassifier
5  import pickle
6
7  df = pd.read_csv('credit_card_data.csv')
8
9  # 1. REMOVE 3 columns + ADD Annual_Income
10 df = df.drop(['Housing_Type', 'Days_Birth', 'Days_Employed'], axis=1, errors='ignore')
11 if 'Annual_Income' not in df.columns:
12     df['Annual_Income'] = df['Age'] * 50000
13
14 le_dict = {}
15 for col in ['Gender', 'Education', 'Marital_Status', 'Employment_Type']:
16     le = LabelEncoder()
17     df[col] = le.fit_transform(df[col])
18     pickle.dump(le, open(f'le_{col}.pkl', 'wb'))
19
20 X = df.drop('Target', axis=1)
21 y = df['Target']
22 print("Final Columns: "; X.columns.tolist()) # Check cheyadaniki
23
24 X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)
25 scaler = StandardScaler().fit(X_train)
  
```

```

PS D:\Credit_Card_Project> python new_model_13.py
Final Columns: ['Age', 'Gender', 'Education', 'Marital_Status', 'Employment_Type', 'Credit_Score', 'Existing_Loan', 'Num_Family_Mem', 'Annual_Income']
13 Feature Model Trained Successfully
PS D:\Credit_Card_Project>
  
```

- **Activity 5.2: Run the application locally using Flask on localhost:5000 and test all functionalities.**

- ❖ **Start Flask Server:** Ran the application using command `python app.py` in the terminal.
- ❖ **Access Application:** Opened browser and accessed the app at <http://127.0.0.1:5000/>.



The screenshot shows a VS Code editor with a project named 'VasudhaAIML'. The file explorer on the left shows a directory structure with files like `app.py`, `create_csv.py`, `new_model_13.py`, `welcome.html`, and `result.html`. The main editor displays the `app.py` file, which contains a Flask application. The code includes a `predict` function that takes form data (name, age, gender, education, marital, own_car, own_real, emp, annual_income, family) and processes it. The terminal at the bottom shows the command `python app.py` being executed, with output indicating that the Flask app is running on `http://127.0.0.1:5000`. A warning message is also visible: "WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead."

- ❖ **Test Form Submission:** Filled the form with sample data like Age, Income, CIBIL and submitted.
- ❖ **Verify Prediction:** Checked that the application correctly shows Approved or Rejected on `result.html`.
- ❖ **Error Testing:** Tested with invalid inputs and different combinations to ensure the app handles all cases properly.

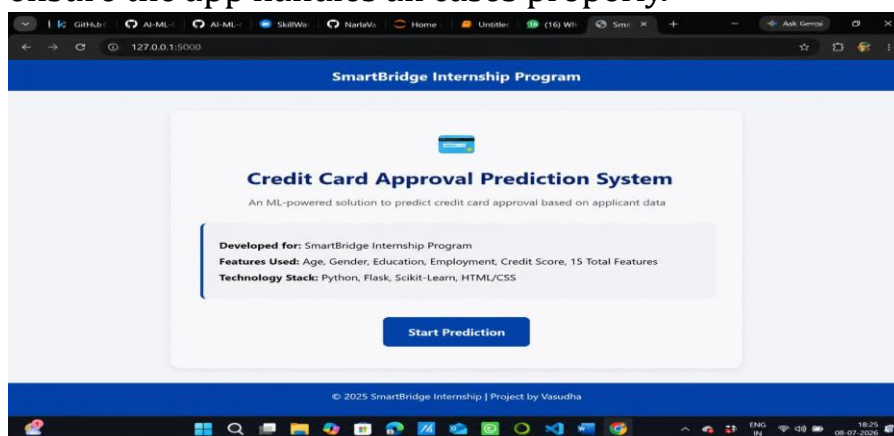
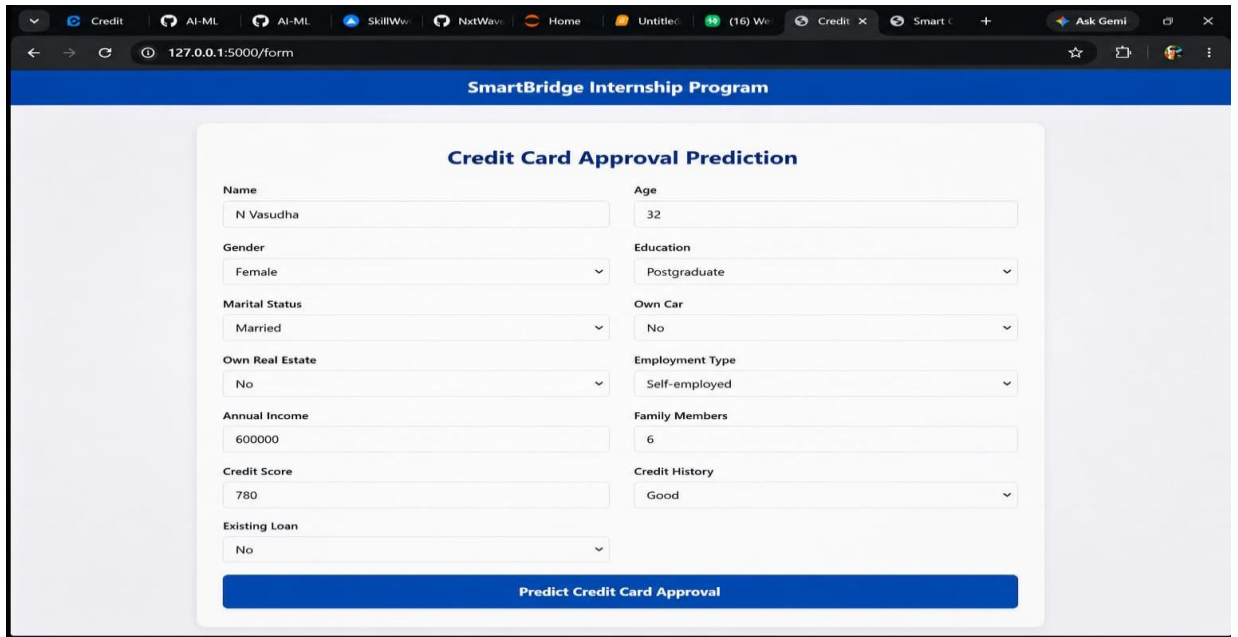


Fig: Welcome Page For Credit Card Approval Prediction System



SmartBridge Internship Program

Credit Card Approval Prediction

Name N Vasudha	Age 32
Gender Female	Education Postgraduate
Marital Status Married	Own Car No
Own Real Estate No	Employment Type Self-employed
Annual Income 600000	Family Members 6
Credit Score 780	Credit History Good
Existing Loan No	

Predict Credit Card Approval

Fig: User Input Form for Credit Card Approval Prediction

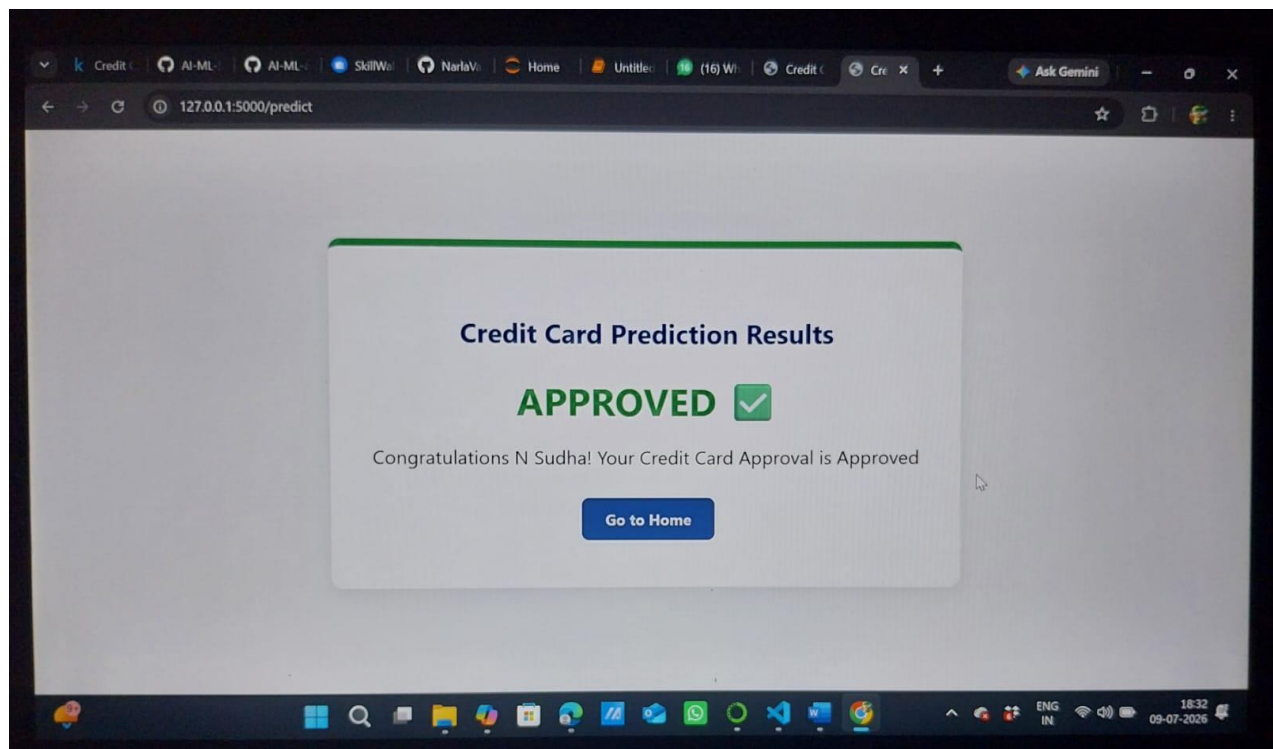


Fig: Prediction Result Page

Milestone 6: Conclusion

This milestone summarizes the overall project outcomes and reflects on the development process of the Credit Card Approval Prediction system.

- **Activity 6.1: Summarize the project outcomes and document the entire development process.**

- ❖ **Project Goal Achieved:** Successfully developed a Machine Learning based system to predict Credit Card Approval using user details like Age, Income, Credit Score.
- ❖ **Model Integration:** Trained and saved a Random Forest model as model.pkl. Integrated it with Flask web framework for real-time predictions.
- ❖ **User-Friendly Interface:** Designed a simple and responsive frontend using HTML and CSS. Used Flask's render_template to dynamically display Approved or Rejected results.
- ❖ **Practical Application:** The system helps banks and financial institutions automate the approval process, saving time and reducing manual errors.
- ❖ **Learning Outcome:** Gained hands-on experience in Data Preprocessing, Model Training, Pickle Serialization, and Flask Web Deployment for real-world ML applications.

Conclusion:

In conclusion, the Credit Card Approval Prediction project demonstrates how Machine Learning can be effectively used in the banking sector. By combining a trained ML model with a Flask web application, we created a system that provides instant and accurate approval decisions. The project was completed successfully by following all milestones from data collection to deployment.